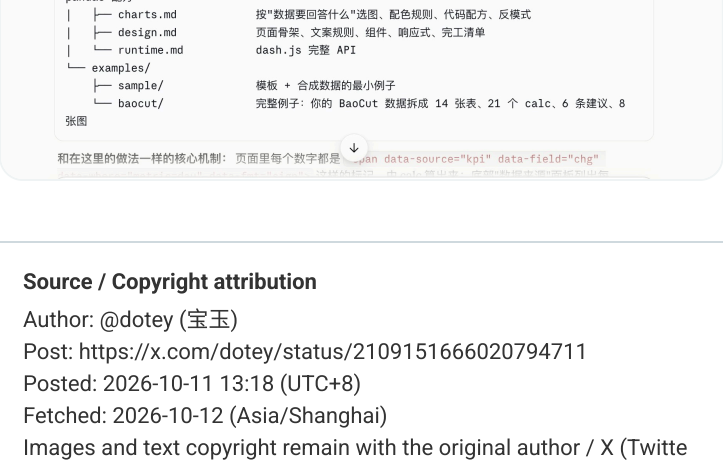
宝玉
@dotey
2026-10-11 13:18 (UTC+8)

用 REA 小试牛刀逆向了 Claude Slides 的 Slides 编辑器，几乎完整功能。（图1） 我的用法很简单，不一定是最佳实践，仅供参考： 1. 先去 Claude Slides 上把核心的脚本和样式表下载下来（图2） 2. 按照 REA 官网的说明让 Codex 安装 REA 3. 使用 REA Skill 去逆向混淆后的代码（图3） 4. 用不了太久就能有一个目录结构不错的代码，但其实里面还挺乱的 5. 然后再用 Claude Code 5.5 去重写了一遍就变成结构良好类型完整的代码了 至于提示词，我没有直接逆向，而是换了个思路，先做个 Slides，然后在对话里面，说我要做一个 slides skill，让它帮我实现一个，要做的跟它的一样好，这样子出来的提示词，和原始的提示词肯定不一样，但是做出来的效果没什么差别。（图4是我用同样方法问 Claude Dashboard 的提示词） 完整的代码参考：https://github.com/JimLiu/baocut/commit/b8585e0c8d7a82bdee9f0ed019cf6edb075e41c1





Yesterday 10:56 AM

Use REA，把 [redacted]slides-type/artifact-type/app.js 代码完整还原成可读性良好的 TypeScript 代码，放到 src/slides-type 目录下 第三方库不需要还原，直接使用 npm 即可

Sent as goal

Activity

我会使用 reverse-engineer-anything 技能分析这个打包文件，把业务代码还原到 `src/slides-type`，并将第三方依赖替换为 npm 包。

Loaded a tool, ran commands

Library inventory finished

Read files, ran commands

Shared names finished

Read files, ran commands

原包约 3.3 MB，包含 5,769 个顶层语句，且没有附带 source map。我正在清点库边界和业务模块；会保留语句覆盖清单，并用原包作为对照验证关键行为。

Ran sed -n '2150,2220p' /tmp/slides-inventory.txt && sed -n '2900,2960p' /tmp/slides-inventory.txt && s...

Model names, Editor names and Sync names started working

Read files, ran commands

Worked for 35m 26s

已还原到 src/slides-type，共 288 个 TypeScript 模块。第三方使用 npm，PPTX 自有扩展通过可复现补丁保留。

类型检查、构建、17 项自动测试和 16 个 PPTX 导出对照均通过，重新生成结果一致。

原包没有 source map，名称和类型根据代码推定。用时约 37 分钟。

Edited 8 files +317 -6 Undo View changes

src/slides-type/globals.d.ts +4 -0

src/slides-type/tools/assemble-groups.mjs +46 -0

src/slides-type/virtual-dependencies.ts +10 -0

Show 5 more files

我正在写一个 Dashboard 这样的 skill，目标是像你一样，输入数据能生成美观直观可交互的 dashboard，请帮我创建这样的 skill，包括 scripts 和 references，就像是你的一个克隆版本，这样我本地也能运行。

Planning the local skill folder with scripts and reference docs.

我来把这套做看板的流程整理成一个可以本地运行的 skill：SKILL.md + 可复用的运行时（dash.js / dash.css）+ 脚本（数据画像、打包、校验）+ 参考文档（分析方法、图表选型、设计规范）。先看一下 skill 的规范。

Created 12 files, edited 2 files, and 29 more steps +1 [redacted]

做好了，`data-dashboard.skill` 和 `data-dashboard.zip` 内容相同：前者在卡片上点"保存"就能装进 Claude；后者解压到本地 `~/.claude/skills/`（或你的项目 `.claude/skills/`）就能在 Claude Code 里用。

结构

```
data-dashboard/
├── SKILL.md                  5 步流程：画像 → pandas 分析 → 写页面 → 检查/打包/截图 → 交付
├── assets/
│   ├── dash.js               零依赖运行时（~500 行）：CSV 解析、calc、data-source 标记、
│   │                         柱/堆叠/折线/mini list/漏斗/可排序表格、tooltip、数据来源面板、主
题切换
│   ├── dash.css              浅色+深色两套 token（沿用 dataviz 的校验过的配色）和全部组件样式
│   └── template.html         页面骨架：标题结论、KPI、建议卡、板块、Sources
├── scripts/
│   ├── profile_data.py       数据画像：时间列/粒度/导出时间、长表拆分建议、维度和指标
│   ├── build.py              把 css+js+CSV 全部内联成一个 HTML，双击就能打开，不用服务器
│   ├── check.py              静态检查：引用了不存在的 calc、正文里没走标记的数字、JS 语法、重复
id
│   └── smoke.py              无头 Chromium：console 报错、没加载出来的骨架、手机宽度溢出、三张截
图
├── references/
│   ├── analysis.md           从文件到结论的方法：标准切法、验证、怎么写建议、必须声明的口径、
pandas 配方
│   ├── charts.md             按"数据要回答什么"选图、配色规则、代码配方、反模式
│   ├── design.md             页面骨架、文案规则、组件、响应式、完工清单
│   └── runtime.md            dash.js 完整 API
└── examples/
    ├── sample/               模板 + 合成数据的最小例子
    └── baocut/               完整例子：你的 BaoCut 数据拆成 14 张表、21 个 calc、6 条建议、8
张图
```

和在这里的做法一样的核心机制： 页面里每个数字都是 `<span data-source="kpi" data-field="chg"`

Source / Copyright attribution

Author: @dotey (宝玉)
Post: https://x.com/dotey/status/2109151666020794711
Posted: 2026-10-11 13:18 (UTC+8)
Fetched: 2026-10-12 (Asia/Shanghai)
Images and text copyright remain with the original author / X (Twitter).
Archived for personal reference; do not redistribute without permission.